

Table of Contents

1.1 Project Objectives.....	1
1.1.1 Objective 1 — Multi-Entity Matrix Intersection Engine.....	1
1.1.2 Objective 2 — Optimistic Concurrency Control (OCC) Layer.....	2
1.1.3 Objective 3 — First-Come-First-Served (FCFS) Digital Approval Dashboard.....	3
1.1.4 Objective 4 — Role-Based Access Control (RBAC).....	4
1.1.5 Objective 5 — Prototype Deployment.....	4
1.2 Problem Statement.....	5
1.2.1 Fragmented Process and Human Error.....	5
1.2.2 Blind Spots in Multi-Cohort Scheduling.....	6
1.2.3 Venue Availability Tracking.....	6
1.2.4 Data Race Conditions.....	6
1.2.5 Elevated Administrative Burden on Programme Leaders.....	7
1.3 Project Background.....	7
1.4 Advantages and Contributions.....	9
1.4.1 Advantages (over Google Sheets).....	9
1.4.2 Contributions (Stakeholders, SDG, Commercialisation).....	11
1.5 Project Plan.....	12
1.5.1 Functional Modules.....	12
1.5.2 Milestones (Gantt Table).....	13
1.5.3 Software Development Model — Agile (Iterative Approach).....	14
1.6 Chapter Summary and Evaluation.....	15

1.1 Project Objectives

Opening paragraph (1–2 sentences):

This section defines five project objectives formulated using the SMART framework — Specific, Measurable, Achievable, Relevant, and Time-bound within the FYP1–FYP2 timeline. Each objective is linked to a distinct technical contribution that distinguishes this system from generic scheduling solutions.

1.1.1 Objective 1 — Multi-Entity Matrix Intersection Engine

Statement:

To design and implement a Multi-Entity Matrix Intersection Engine that computes valid replacement windows by performing a four-way set intersection across lecturer availability, stacked cohort free schedules (supporting mixed-cohort groupings), room occupancy vectors, and capacity-aware venue filtering that excludes rooms with insufficient seating.

Why it scores Creativity & Innovation (7 marks):

- **New/unique:** No existing system at TAR UMT Sabah performs automated 4-vector intersection for class replacement. Spreadsheets require manual overlay.
- **Fresh approach:** Uses deterministic set intersection (not heuristic or ML-based), making results predictable and auditable.
- **Leverages IT trends:** Relational set operations on real-time timetable data, a modern alternative to static cross-referencing.

Measurable success criteria:

- Engine returns correct common free slots for any combination of multiple cohorts within 500ms
 - Capacity filter correctly excludes rooms smaller than total enrolled student count
 - **Handles edge cases:**
 - **No common slot found** — a lecturer wants a replacement but the engine finds zero overlapping free time across lecturer, cohorts, and room. The system should return "No available slots" rather than crashing or returning incorrect results.
 - **Single-cohort request** — a replacement affects only one cohort (not mixed). The engine should still work correctly with just 3 vectors (lecturer $\times$ 1 cohort $\times$ room) instead of 4.
 - **All-day occupancy** — the lecturer is fully booked the entire day with no free slots. The engine should correctly return empty results rather than showing occupied slots as available.
 - **Session-type venue restrictions** — When scheduling a module with specific delivery modes (e.g., MPU-3133 which is Lecture-only), the engine must apply constraints that filter strictly for Lecture Halls (e.g., B110, B111) and automatically exclude Labs or Tutorial rooms regardless of open slots.
-

1.1.2 Objective 2 — Optimistic Concurrency Control (OCC) Layer

Statement:

To implement an Optimistic Concurrency Control (OCC) layer that prevents double-booking by performing a millisecond-precision transactional validation at submission time, automatically aborting and rolling back if a concurrent transaction has modified the slot state.

Why it scores Creativity & Innovation:

- **New/unique:** First application of OCC (rather than pessimistic locking) for class replacement at the faculty. Spreadsheets offer zero transaction support.
- **Fresh approach:** OCC avoids performance-heavy database locks, keeping the UI responsive even under concurrent lecturer submissions.

- Leverages IT trends: ACID-compliant transactional design in a web application context, referencing Kung & Robinson's foundational work.
- "The OCC layer operates within Atomicity, Consistency, Isolation, Durability (ACID) database transactions to guarantee data integrity under concurrent access."

Measurable success criteria:

- Two simultaneous submissions for the same slot → exactly one succeeds, one receives rollback alert
 - Validation checks slot state at the millisecond of database write
 - All transactional outcomes logged in audit trail
-

1.1.3 Objective 3 — First-Come-First-Served (FCFS) Digital Approval Dashboard

Statement:

To develop a FCFS digital approval dashboard that automates the Programme Leader's (PL) verification workflow — replacing the manual rechecking of schedules, venue cross-referencing, and "Y" annotation process — with a pre-validated queue that enables one-click approval (slot transitions to Occupied) or rejection with mandatory reason (slot returns to Available), with every action recorded in a timestamped audit trail.

Why it scores Creativity & Innovation:

- **New/unique:** Currently PLs must manually recheck every request against lecturer schedules, cohort timetables, and venue in PDFs before writing "Y" in a spreadsheet cell. The dashboard eliminates this rechecking entirely because the engine has already validated the slot.
- **Fresh approach:** One-click approval replaces a multi-step manual verification pipeline. The audit trail is automatic rather than retroactively reconstructed.
- **Leverages IT trends:** State machine design (Available → Pending → Occupied/Rejected) for structured workflow management.

Measurable success criteria:

- Queue displays all pending requests sorted by submission timestamp (earliest first — matching current chronological order)
- Each request shows pre-computed slot validity so PL does not need to re-verify
- Approval: 1 click → slot becomes Occupied, all future requests for that slot auto-blocked
- Rejection: requires mandatory text reason → slot returns to Available, available for rebooking
- Audit trail records: PL identity, timestamp, action taken, slot ID, reason (if rejected)

1.1.4 Objective 4 — Role-Based Access Control (RBAC)

Statement: To build a role-based access control system with three tiers:

- view-only access for students,
- request-and-submit workflows for lecturers, and
- hybrid administrative privileges for Programme Leaders who retain full lecturer creation rights in addition to queue management.

Why it scores Creativity & Innovation:

- **New/unique:** Hybrid PL role (admin + lecturer creation rights) is a practical design not commonly found in generic scheduling tools. Existing Google Sheets have no role differentiation — anyone with the link can edit. The hybrid PL role (admin + lecturer rights) reflects the real FOCS structure where PLs also teach.
- **Fresh approach:** Designed around actual FOCS workflow rather than generic RBAC templates. — PLs are also teaching staff, so they need both privilege sets.
- **Leverages IT trends:** Laravel's built-in authorization gates and middleware for declarative permission checks.

Measurable success criteria:

- Student: can view timetables and replacement status only (no create/edit/delete)
 - Lecturer: can create and submit requests, edit or cancel own pending requests , cannot approve or view other lecturers' requests
 - PL: inherits all lecturer rights + can approve/reject any request in queue
 - Authentication via username/password with session management
 - Unauthenticated users: cannot access any dashboard page (redirected to login)
-

1.1.5 Objective 5 — Prototype Deployment

Statement:

To deploy a fully functional Laravel-PostgreSQL web application on a local development server, seeded with :

- **Eleven (11) total student cohorts consisting of:**
 - **FOCS:** DFT1(S1), DFT2(S1), DSF1(S1), DSF2(S1), RSD1(S1)G1, RSD2(S1)G1, G2, G3, and RSD3(S1)G1, G2, G3
 - **FAFB:** RAF2(S3)G2, G4 and RBU1(S1)G1
- **14 total academic staff across 3 Sabah departments:**
 - 11 DCIT staff (9 Lecturers + 2 Programme Leaders)
 - 2 DACB Lecturers
 - 1 DSSH Lecturer
- 20 classroom venues (B002, B005, B006, B009–B011, B014–B018 , B101–B111)

- for prototype validation within the FYP2 sprint phase.

Why it scores Feasibility (5 marks) + supports Creativity:

- **Developable within FYP timeframe:** cohorts, 14 staff, 20 rooms — realistic scope for 7-week sprint
- **Relevant to the problem:** Demonstrates the engine working with real-world scale data, not toy data
- **Technology:** Laravel (PHP), PostgreSQL, TailwindCSS, Blade templating — modern full-stack stack

Measurable success criteria:

- All seed data loads correctly (no missing foreign keys)
 - Application runs on local development server (Laravel Artisan serve)
 - All three user roles can log in with test credentials and access their respective dashboards
 - Each objective's success criteria individually verifiable through test cases
-

1.2 Problem Statement

Opening paragraph:

This section identifies five interrelated weaknesses in the current class replacement process at FOCS Sabah. Each problem is supported by academic literature and linked to specific consequences that motivate the development of the proposed system.

1.2.1 Fragmented Process and Human Error

Statement:

The management of replacement requests relies on a decentralised web of Google Sheets where each lecturer maintains their own schedule independently. No centralized database exists to unify the state of replacements across the faculty, requiring numerous manual cross-referencing steps for each request (Babaei, 2015).

Why it reflects a real-world problem:

- Each lecturer keeps a personal schedule spreadsheet with no shared source of truth
- No central database to track replacement state across the cohorts, creating information silos between different cohorts.
- Every replacement request requires manual cross-referencing of multiple spreadsheets

Supporting citation:

Babaei, Karimpour, & Hadidi (2015) established that manual scheduling processes in educational organisations significantly increase the probability of human error.

Consequence: Scheduling mismatches that require downstream correction, disrupting both lecturers and students.

1.2.2 Blind Spots in Multi-Cohort Scheduling

Statement:

When a module draws students from multiple cohorts simultaneously (e.g., BMSE3153 shared by RSD3G1,RSD2G2,RSD2G3), manually combining two or more independent cohort schedules to find common free periods is both time-consuming and error-prone.

Why it reflects a real-world problem:

- Each cohort follows a distinct module timetable
- Common free period intersection is not immediately visible
- Manual spreadsheet overlay of separate timetable files introduces mistakes
- Errors are only discovered after replacement has been scheduled

Consequence:

Incorrect replacement slots that require further rescheduling and cause disruption to teaching activities.

1.2.3 Venue Availability Tracking

Statement:

Information regarding venue availability is stored separately from the scheduling process as a static PDF file. Lecturers must independently check whether classrooms are free for replacement, and there is no automated capacity-aware filtering.

Why it reflects a real-world problem:

- Room availability tracked in isolation from scheduling logic
- No capacity-aware filtering to prevent assigning a room too small for combined cohorts
- Room usage priority rules (Lab only for P, B006 priority for Networking) not enforced

Consequence:

Venue conflicts discovered late, requiring additional rescheduling and administrative overhead.

1.2.4 Data Race Conditions

Statement:

The collaborative nature of shared Google Sheets offers no transaction boundaries. When multiple lecturers submit requests simultaneously, both see the same slot as available and proceed, creating an irresolvable double-booking conflict.

Why it reflects a real-world problem:

- Google Sheets has no transaction isolation between concurrent users

- Two lecturers submitting at the same time both see the slot as available
- Double-booking only discovered after both requests are processed
- No rollback mechanism exists to recover from the conflict

Supporting citations:

Al-Hawari, Al-Ashi, Abawi, & Alouneh (2020) — absence of concurrency control in multi-user scheduling leads to data integrity violations. Kung & Robinson (1981) — optimistic concurrency control methods can resolve such conflicts without the performance overhead of pessimistic locking.

Consequence: Irresolvable booking conflicts requiring Programme Leader intervention and manual reconciliation.

1.2.5 Elevated Administrative Burden on Programme Leaders

Statement:

Programme Leaders must manually verify every incoming request against lecturer schedules, cohort timetables, and venue availability records before annotating approval in the spreadsheet. Each verification is performed from scratch with no automated decision support.

Why it reflects a real-world problem:

- PL manually checks each request against lecturer schedules
- PL manually cross-references cohort timetables (often multiple cohorts)
- PL manually checks venue availability from separate PDF
- All verification is performed with no automated decision support

Supporting citation:

Schaerf (1999) — centralising scheduling logic into a dedicated system eliminates the redundancy and error propagation inherent in manual coordination.

Consequence: Replacement processing takes hours or days instead of minutes, and the manual verification process itself introduces further opportunity for oversight.

Closing paragraph:

The absence of a centralised, transaction-aware scheduling platform at FOCS Sabah has resulted in recurring scheduling conflicts, prolonged replacement processing times, and an opaque audit trail, necessitating the development of an automated replacement scheduling system.

1.3 Project Background

Here is your completely updated, polished **Section 1.3 (Project Background)** point-form draft. It integrates your exact TARUMT Sabah administrative reality, your new

11-cohort dataset (replacing the old 5-cohort scope), your precise room matrix, your department breakdown, and the project team hierarchy.

1.3 Project Background

1.3.1 Institutional, Structural, and Administrative Context

- **TAR UMT Sabah Campus Domain:** - The system is primarily engineered for the **Faculty of Computing and Information Technology (FOCS)** at the Sabah Branch.
 - Cohort-based delivery structure handles computing tracks spanning Diploma (**DFT**, **DSF**) and Degree (**RSD**) levels.
- **The Faculty vs. Department Structural Split:**
 - **Programmes & Cohorts** are strictly bounded by **Faculties**: FOCS acts as the primary system client, but the system must account for structural dependencies involving 3 external cohorts from the Faculty of Accountancy, Finance and Business (**FAFB**).
 - **Academic Staff** are organized strictly by specialized teaching **Departments** rather than faculties.
- **Academic Staff Composition (Sabah Branch Workloads):**
 - Total resource pool comprises **14 lecturers spanning 3 foundational Sabah departments**:
 - **Department of Computing and Information Technology (DCIT):** 11 staff total, consisting of 9 Lecturers and 2 Programme Leaders (**PLs**) who retain active teaching workloads.
 - **Department of Accountancy, Finance and Business (DACB):** 2 Lecturers handling business/entrepreneurship modules.
 - **Department of Social Science and Humanities (DSSH):** 1 Lecturer handling general education/MQA national modules.
 - *The Administrative Challenge:* Because **DSSH** and **DACB** staff teach modules across FOCS cohorts, schedule coordination naturally cuts across separate departmental tracking environments.

1.3.2 Prototype Comprehensive Dataset Specification

- **Full Cohort Structural Dataset (11 Target Groups):**
 - Instead of a limited single-programme track, the prototype database is seeded with a rigorous multi-programme, multi-faculty baseline comprising **11 active cohorts**:
 - **FOCS Cohorts (8 groups):** **DFT1(S1)**, **DFT2(S1)**, **DSF1(S1)**, **DSF2(S1)**, **RSD1(S1)G1**, **RSD2(S1)G1**, **RSD2(S1)G2**,

RSD2(S1)G3, RSD3(S1)G1, RSD3(S1)G2, and RSD3(S1)G3.

- **FAFB Testing Cohorts (3 groups):** RAF2(S3)G2, RAF2(S3)G4, and RBU1(S1)G1 (strategically seeded to validate cross-faculty MPU intersection dependencies).

- **Core Master Timetable Data Profile:**

- **Source Material:** Real-world past semester timetables exported directly from the campus *aSc Timetables* planning utility.
- **Timeline Scope:** Maps an identical, fixed, repeating weekly baseline structure across a standard 14-week long academic semester (focusing on static schedules to isolate replacement calculations).

1.3.3 Physical Venue Infrastructure Matrix

- **Prototype Dataset Venue Scope (20 Bounded Rooms):**

- To secure a realistic data volume while keeping development feasible within the 7-week sprint, the engine isolates its tracking strictly to **20 high-density rooms located in Block B:**
 - **Tutorial Rooms (Capacity ≤ 35, standard L/T sessions):** B101, B102, B103, B104, B105, B106, B107, B108, B109, B014, B015, B016, B017, B018, and B002.
 - **Lecture Halls (Capacity > 35, large multi-cohort assemblies):** B110 and B111.
 - **Computer Labs (Capacity = 28, Practical-only 'P' sessions):** B005, B009, B010, and B011.
 - **Cisco Specialized Lab (Capacity = 32, Priority for Networking & IoT workloads):** B006.
- *Operational Boundary Exclusion:* Venues situated in Block C are explicitly omitted from the seeding profile to strictly guard the validation boundaries.

1.3.4 High-Risk Matrix Intersection Scenarios (The Rationale)

The expanded 11-cohort dataset forces the proposed engine to calculate complex, high-risk scheduling interdependencies that manual spreadsheets routinely overlook:

- **Use Case 1: Cross-Faculty Synchronization (MPU-3133 Falsafah dan Isu Semasa)**

- *Syllabus Pattern:* A Lecture-only (L) module shared simultaneously across RAF2, RBU1, and RSD3.
- *Engine Constraint:* The engine must compute a unified open slot across different faculties while automatically applying a venue-layout filter that locks the session strictly to B110 or B111, rejecting smaller tutorial rooms

based on combined student volume.

- **Use Case 2: Cross-Year Internal Stacking (MPU-3232 Entrepreneurship)**
 - *Syllabus Pattern:* A Lecture & Tutorial (L&T) module shared horizontally between different academic years within FOCS (RSD2(S1)G2, RSD2(S1)G3, and RSD3(S1)G3).
 - *Engine Constraint:* The engine must calculate a set intersection that prevents horizontal clashing, ensuring a replacement for one cohort does not step on the concurrent core module constraints of the other stacked year.

1.3.5 Current Replacement Workflow (Manual)

- **Decentralised Execution Architecture:** Handled completely via a web of shared Google Sheets with zero centralized relational linking.
- **The Multi-Step Manual Pipeline:**
 1. The lecturer notices a replacement requirement and consults an isolated personal timetable.
 2. Manually cross-references the independent timetable PDFs/spreadsheets of all affected cohorts to calculate overlapping open periods.
 3. Checks physical venue availability by eye using a standalone, static campus room occupancy PDF.
 4. Manually drafts the request details and submits them to the Programme Leader, waiting for asynchronous validation and administrative sign-off.
- *The Failure Point:* When a module involves multiple or cross-faculty cohorts, the manual overlay of separate data files becomes incredibly error-prone.

1.3.6 Why the Current Process is Insufficient

- **Information Silos:** No unified transactional database exists; each lecturer tracks their own replacement operations in isolation.
- **Manual Mathematics:** Calculating overlapping common free windows across diverse student groups is done completely by hand with zero algorithmic support.
- **Data Detachment:** Room occupancy vectors are tracked entirely separately from the actual scheduling engine.
- **Zero Race-Condition Protection:** Google Sheets offers no transactional isolation boundaries, allowing two concurrent lecturers to claim the exact same classroom slot simultaneously without throwing a database error.
- **Administrative Redundancy:** Programme Leaders are forced to rebuild the entire verification process from scratch for every single incoming request—re-checking lecturer workloads, cohort times, and venue capacities with no automated decision support.

- *Downstream Impact:* Processing a single schedule adjustment degrades into an administrative bottleneck taking hours or days instead of minutes.

1.3.7 Project Team and Governance Structure

- **Developer / Investigator:** Poong Foo Jing (3rd-Year FOCS Software Systems Development Student).
 - **Academic Guidance Hierarchy:** Under the direct FYP supervision of **Mr. Lim Jia Zheng**, with **Ms. Teng Nga Sing** serves as the official FYP moderator.
 - **Domain Expertise & Requirements Validation Panel:** Structural logic, administrative rules, and database constraints are validated directly by two FOCS Programme Leaders:
 - **Pn. Surayaini Binti Basri**
 - **En. Mohd Nur Rahmat Bin Mohd Taat**
 - **Primary Project Client:** Faculty of Computing and Information Technology (FOCS), TAR UMT Sabah Branch.
-

1.4 Advantages and Contributions

Opening paragraph:

This section outlines the advantages of the proposed TARUMT Class Replacement System over the existing Google Sheets-based workflow, followed by its contributions to stakeholders, the institution, and alignment with the United Nations Sustainable Development Goals.

1.4.1 Advantages (over Google Sheets)

Advantage 1 — Eliminates Manual Cross-Referencing

- Google Sheets: lecturers manually overlay separate timetable files to find common free periods
- Proposed system: Matrix Intersection Engine computes valid slots automatically in milliseconds
- Benefit: reduces replacement processing from hours/days to minutes

Advantage 2 — Prevents Double-Booking

- Google Sheets: no transaction boundaries → simultaneous submissions cause conflicts
- Proposed system: OCC validates each submission at database-write level → double-booking impossible
- Benefit: eliminates administrative overhead of resolving booking conflicts

Advantage 3 — Automated Venue Verification

- Google Sheets: lecturers check venue availability from separate static PDF

- Proposed system: capacity-aware room filtering is integrated into the intersection engine
- Benefit: venues with insufficient seating are automatically excluded

Advantage 4 — Centralised Replacement State

Google Sheets (current):

- Each lecturer maintains their own personal schedule Google Sheet
- There is no central record of which replacement requests exist, who submitted them, or their current status
- If Lecturer A submits a request, Lecturer B has no way of knowing unless the PL manually communicates it
- The state of a slot (free, pending, approved, occupied by another booking) is not tracked anywhere
- At any given time, no single person has a complete picture of all replacements happening in the faculty

Proposed system:

- All timetable data, replacement requests, and slot states live in a single PostgreSQL database
- Every slot has an explicit state:

State	Meaning
Available	Slot is free — no conflict exists, no replacement pending
Pending (Self)	The logged-in lecturer has submitted a request for this slot, awaiting PL approval
Reserved (Other)	Another lecturer has submitted a request for this slot (FCFS priority applies)
Occupied	PL has approved the replacement — slot is locked permanently

Benefit:

- Anyone logging into the system instantly sees the real-time state of every slot across the faculty — no need to ask around or cross-reference separate sheets
- Lecturers avoid wasting time on slots that are already Reserved by another lecturer
- PLs see all pending requests in one place without chasing down email threads
- The database is the single source of truth — no conflicting versions of reality

Advantage 5 — Structured Approval Workflow with Audit Trail

- Google Sheets: PL manually writes "Y" in a cell, no automatic logging
 - Proposed system: FCFS dashboard with one-click approval + full timestamped audit trail
 - Benefit: complete accountability for all replacement decisions
-

1.4.2 Contributions (Stakeholders, SDG, Commercialisation)

Contribution to Lecturers

- Eliminates manual cross-referencing of schedules and venue PDFs
- Provides instant visibility of available slots with clear state indicators
- Reduces time spent on replacement administration

Contribution to Programme Leaders

- Pre-validated queue removes need for manual re-verification
- One-click approval with automatic audit trail
- Clear chronological overview of all pending requests

Contribution to Students

- View-only access to consolidated master timetable
- Real-time visibility of replacement status and schedule changes
- Ensures schedule integrity through OCC prevention of conflicts

Contribution to the Institution (FOCS Sabah)

- First system at TAR UMT Sabah to handle cross-cohort class replacement with OCC
- Establishes a technical precedent for concurrency-aware scheduling applications

Alignment with SDG 4 — Quality Education

- Reduces disruption to teaching schedules through rapid replacement processing
- Ensures continuity of learning activities when replacement is needed

Alignment with SDG 8 — Decent Work and Economic Growth

- Automates administrative coordination, freeing academic staff for teaching and mentoring
- Demonstrates integrity in educational resource management through OCC

Commercialisation and Showcase Potential

Proposed adoption pipeline:

- **Phase 1:** Deploy within FOCS TAR UMT Sabah (initial target — already scoped for this)
- **Phase 2:** Expand to all faculties at TAR UMT Sabah campus
- **Phase 3:** Adoption by TAR UMT main campus (KL) and relevant departments
- **Phase 4:** Roll out across all TAR UMT branches nationwide
- **Phase 5:** Adapt for other universities facing similar scheduling challenges

Showcase opportunities:

- Candidate for TAR UMT Smart Campus Initiative presentation
- Technology showcase at faculty or institutional events
- Reference implementation for OCC-based scheduling in academic environments

1.5 Project Plan

1.5.1 Functional Modules

Opening:

The system is decomposed into five functional modules. Each module encapsulates a distinct set of responsibilities, enabling modular development, testing, and maintenance throughout the FYP1–FYP2 timeline.

Module 1 — Schedule and Timetable Management Module

- Stores and manages timetable data for lecturers, cohorts, and rooms
- **Maintains master records for:** 11 total student cohorts spanning 2 faculties (8 FOCS: DFT, DSF, RSD; 3 FAFB: RAF, RBU), 14 Lecturers spanning 3 academic departments (DCIT, DACB, DSSH), and 20 Block B classrooms.
- Defines venue metadata: capacity, room type (Tutorial/Lecture Hall/Lab/Cisco Lab), allowed session types (L/T/P)
- Provides the data foundation for all other modules
- **Related objective:** 1.1.5 (Deployment)

Module 2 — Multi-Entity Matrix Intersection Module

- Core computational layer implementing the 4-vector set intersection algorithm
- Inputs: lecturer availability, 1+ cohort free schedules, room occupancy (from selected venue in dropdown), capacity filter
- Venue selection: dropdown above the timetable grid, defaulted to the class's **original venue**
- Changing the venue dropdown recalculates available slots in real-time
- Output: a weekly timetable grid (time × day) with color-coded cells:
 - **Green** — slot satisfies all 4 constraints (lecturer × cohort(s) × room × capacity → clickable)
 - **Red** — slot is occupied by an existing class
 - **Yellow** — slot has a pending request you submitted
 - **Grey** — slot is reserved by another lecturer's submission
 - **Blue** — Current Selection
- **Related objective:** 1.1.1 (Matrix Engine)

Module 3 — Optimistic Concurrency Control (OCC) Module

- Transactional validation layer that checks slot state at the precise millisecond of database write
- Handles: concurrent submission detection, rollback on conflict, user-facing alert generation
- Manages the slot state machine: Available ↔ Pending ↔ Reserved ↔ Occupied
- **Related objective:** 1.1.2 (OCC Layer)

Module 4 — FCFS Approval Dashboard Module

- Chronologically ordered queue of all pending replacement requests
- One-click approval (slot → Occupied) and rejection with reason (slot → Available)
- Full audit trail: PL identity, timestamp, action, slot ID, rejection reason
- **Related objective:** 1.1.3 (FCFS Dashboard)

Module 5 — Role-Based Access Control (RBAC) Module

- Authentication (username/password) and session management
 - Three role tiers: Student (view-only), Lecturer (create/submit), PL (hybrid)
 - Authorization gates on all routes and actions
 - **Related objective:** 1.1.4 (RBAC)
-

1.5.2 Milestones (Gantt Table)

Opening:

The development follows an Agile iterative approach across FYP1 (14 weeks) and FYP2 (7 weeks). High-risk features (schedule matching, OCC) are prioritised early.

Phase	Activity	Expected Outcome	Target Date
FYP1	Requirements Elicitation and Problem Formulation	Approved Form 2 Proposal	FYP1 Week 2
FYP1	Chapter 1: Introduction	Completed problem statement, objectives, scope	FYP1 Week 5
FYP1	System Requirements and Use Case Modelling	SRS, use case diagrams	FYP1 Week 6
FYP1	Chapter 2: Literature Review	APA-cited review of OCC, timetabling algorithms	FYP1 Week 8
FYP1	Database Schema Design	Normalised PostgreSQL schema, ERD	FYP1 Week 9
FYP1	Chapter 3: Methodology and Requirements	Functional + non-functional requirements	FYP1 Week 10
FYP1	Chapter 4: System Design	Architecture, UI mockups, OCC pseudocode	FYP1 Week 12
FYP1	Interim Prototype	Routing framework, seed data	FYP1 Week 13

Phase	Activity	Expected Outcome	Target Date
FYP1	FYP1 Portfolio Submission	Chapters 1–4 report, viva presentation	FYP1 Week 14
FYP2	Sprint 1: Matrix Intersection Engine	Functional core API for slot intersection	FYP2 Week 2
FYP2	Sprint 2: OCC + FCFS Dashboard	Conflict prevention, approval queue	FYP2 Week 4
FYP2	Sprint 3: UI Integration + RBAC	Complete interface, test report	FYP2 Week 5
FYP2	Final Thesis Compilation	Complete report (Chapters 5–7)	FYP2 Week 6
FYP2	Final Viva Defence	Presentation + live demo	FYP2 Week 7

1.5.3 Software Development Model — Agile (Iterative Approach)

Opening:

Due to the limited development period, the Agile iterative development method is adopted. This approach supports rapid development through short, repeated improvement cycles, allowing early testing of critical system components.

Why Agile fits this project:

- **Limited timeframe:** FYP2 has only 7 weeks — Agile's time-boxed iterations (sprints) fit naturally
- **High-risk features prioritised early:** The Matrix Intersection Engine and OCC layer are developed in Sprint 1 and Sprint 2 respectively, ensuring the most technically challenging components have the most testing time
- **Incremental delivery:** Each sprint produces a working, testable increment — Sprint 1 delivers the core API, Sprint 2 adds transactional integrity, Sprint 3 completes the UI
- **Adaptability:** Requirements can be refined based on supervisor feedback during progress meetings
- **Continuous validation:** Each increment can be demonstrated and validated before the next sprint begins

Sprint breakdown:

- **Sprint 1 (FYP2 W1–2):** Matrix Intersection Engine — database schema, timetable

import, intersection algorithm API

- **Sprint 2 (FYP2 W3–4):** OCC Layer + FCFS Dashboard — transaction handling, approval queue, audit trail
 - **Sprint 3 (FYP2 W5):** UI Integration + RBAC — Blade views, TailwindCSS styling, role gating, test cases
 - **Sprint 4 (FYP2 W6–7):** Final thesis compilation, system documentation, viva preparation
-

1.6 Chapter Summary and Evaluation

Opening paragraph:

This section summarises the contents of Chapter 1 and provides a self-evaluation of the completeness and feasibility of the project definition.

Summary:

- This chapter established the background and motivation for the TARUMT Class Replacement System at FOCS Sabah
- Five SMART objectives were defined, each specifying a measurable success criterion and a distinct technical contribution
- The problem statement identified five specific pain points in the current manual workflow, supported by academic literature
- The system was decomposed into five functional modules with clear boundaries, defining the project scope
- Five advantages over the existing Google Sheets workflow were presented, followed by stakeholder contributions including SDG 4 and SDG 8 alignment
- A phased project plan was provided across FYP1 (14 weeks) and FYP2 (7 weeks), following an Agile iterative approach with three development sprints

Self-Evaluation:

Objectives:

- All five objectives are clearly measurable — each has specific success criteria verifiable through test cases
- Each objective maps to a distinct rubric criterion (Creativity & Innovation, Feasibility, Contribution & Reach)

Scope:

- Scope is explicitly defined through functional module decomposition — no ambiguity about what will be built
- Dataset boundaries are precise: 11 total cohorts spanning 2 faculties (8 FOCS, 3 FAFB), 14 lecturers spanning 3 administrative departments (DCIT, DACB, DSSH), and 20 Block B rooms.

- Excluded venues **and campus spaces (such as Block C classrooms)** are explicitly listed **to isolate and protect the prototype sprint timeline.**

Problem Statement:

- The problem is distinguished from generic scheduling solutions by focusing on multi-cohort set intersection and Optimistic Concurrency Control, rather than attempting broader university timetabling

Feasibility:

- The phased schedule allocates appropriate time for design (FYP1) and implementation (FYP2)
- The 4-sprint structure with high-risk features prioritised early ensures critical components have the most testing time
- The expanded cross-faculty dataset scale remains entirely appropriate and realistic for a prototype validation within the 7-week FYP2 implementation sprint window. *(Update this line)*